

DATA-DRIVEN DECISION MAKING
IN OPERATIONS RESEARCH

Problem Report

x x

October 2, 2026

*Student:*Hengyi Li
Binghua Yao*Supervisor:*

Dr. A. V. den Boer

Problem Statement

Consider a multi-armed bandit with $k = 2$ arms and $n = 200$ time periods. Rewards $X_{t,a}$ are uniformly distributed with support $[l_a, u_a]$, for unknown $0 \leq l_a < u_a \leq 1$, $a = 1, 2$.

- (a) Design a good possible policy based on your own ideas: determine for each $t = 1, \dots, n$ a (possibly random) action A_t based solely on the available data $(A_s, X_s)_{s \leq t-1}$, and not on future data or on the true values of μ_a . Carefully describe the underlying ideas and intuition, and why you believe this is a good policy. Give both an intuitive and a formal definition of the policy. Try to have the best possible performance.
- (b) Implement the policy and report its instance-dependent regret after n time periods, for all 100 feasible values of $l_1, u_1, l_2, u_2 \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, including convincing confidence bounds. *Hint:* to speed up implementation you may use Additional Exercise 2.
- (b2) As a sanity check, also implement both fixed-action policies that use the same action in all time periods. Report their regret for all feasible values of $l_1, u_1, l_2, u_2 \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, including confidence bounds. Explain whether the observed values conform to what you would expect.
- (c) What is the worst-case regret that you observe, and at which value of l_1, u_1, l_2, u_2 is it attained? Explain the intuition.

Contents

1	Introduction	2
2	Methodology	2
2.1	Policy Design	2
2.2	Simulation	3
2.3	Evaluation	4
3	Conclusion	6
A	Codes	6

1 Introduction

In this report, we examine a two-armed bandit with $n = 200$ time periods. The rewards $X_{t,a}$ follow a uniform distribution with the support $[l_a, u_a]$ for unknown $0 \leq l_a < u_a \leq 1$. Our objective is to devise an effective policy that maximises the cumulative rewards. Furthermore, we assess the efficacy of this policy by computing its regret. For the methodology of this report, we predominantly reference *Bandit Algorithms* by Tor Lattimore and Csaba Szepesvári [1].

2 Methodology

In this section, we delineate the principal methodologies. Initially, we proffer both an intuitive and a formal definition of the policy. Subsequently, we implement the policy and describe its instance-dependent regret after n time periods. Finally, we expound upon the worst-case regret observed.

2.1 Policy Design

Given that for each $t = 1, \dots, n$, an action A_t (which may be random) is determined exclusively based on the extant data $(A_s, X_s)_{s \leq t-1}$, eschewing reliance on prospective data or the true expectation values μ_a , it becomes cogent for us to form an intuition of selecting both arm one and arm two for a specified number of times, denoted as m times ($2m$ times in total), where the exact value is to be ascertained later. This approach is rationalised by the fact that the distributions of ν_1, ν_2 remain undisclosed, thereby necessitating an initial acquisition of empirical data.

Let us start by setting the set $\mathcal{A} = \{1, 2\}$, and $\nu_a = U[l_a, u_a]$, $a = 1, 2$. By the characteristics of the uniform distribution, it follows that

$$\mu_1 = \frac{l_1 + u_1}{2}, \quad \mu_2 = \frac{l_2 + u_2}{2}, \quad 0 \leq l_a < u_a \leq 1, \quad a = 1, 2.$$

We employ the sample mean $\hat{\mu}_1, \hat{\mu}_2$ to estimate μ_1 and μ_2 , respectively, and revise these estimates subsequent to each iteration. Then the determination of the forthcoming action is predicated upon the ratio of these two estimators.

Without loss of generality, we initially choose arm one for m times, subsequently selecting arm two for m times. Given that the set $\mathcal{A} = \{1, 2\}$ is finite, we simply write $\pi(a \mid h)$ instead of $\pi(\{a\} \mid h)$ for the conditional probability that $A_t = a$. For the first $2m$ actions, we have

$$\begin{aligned} \pi(1 \mid h) &= \mathbb{P}(A_t = 1 \mid (A_1, X_1, \dots, A_{t-1}, X_{t-1}) = h) = 1, \quad 0 < t \leq m, \\ \pi(2 \mid h) &= \mathbb{P}(A_t = 2 \mid (A_1, X_1, \dots, A_{t-1}, X_{t-1}) = h) = 1, \quad m < t \leq 2m. \end{aligned}$$

After that, we can propose the formal definition of two estimators:

$$\hat{\mu}_1(t) = \frac{\sum_{i=1}^t X_i \mathbb{1}_{\{A_i=1\}}}{\sum_{i=1}^t \mathbb{1}_{\{A_i=1\}}}, \quad \hat{\mu}_2(t) = \frac{\sum_{i=1}^t X_i \mathbb{1}_{\{A_i=2\}}}{\sum_{i=1}^t \mathbb{1}_{\{A_i=2\}}}, \quad 2m \leq t \leq n.$$

Thereafter, the selection procedure for $2m < t \leq n$ steps is:

$$\begin{aligned} \pi(1 \mid h) &= \mathbb{P}(A_t = 1 \mid (A_1, X_1, \dots, A_{t-1}, X_{t-1}) = h) = \frac{\hat{\mu}_1(t-1)}{\hat{\mu}_1(t-1) + \hat{\mu}_2(t-1)}, \\ \pi(2 \mid h) &= \mathbb{P}(A_t = 2 \mid (A_1, X_1, \dots, A_{t-1}, X_{t-1}) = h) = \frac{\hat{\mu}_2(t-1)}{\hat{\mu}_1(t-1) + \hat{\mu}_2(t-1)}. \end{aligned}$$

If both sample means are zero, we set $\pi(1 \mid h) = \pi(2 \mid h) = 1/2$ to avoid division by zero. As the sample means are updated, the arm with the larger estimated mean is selected with higher probability; the other arm can still be selected. The paramount question thus transitions to the determination of m .

2.2 Simulation

We undertake an examination to ascertain the best m value among those tested within the policy. For $m \in \{1, 2, \dots, 20\}$, we compute the average regret for all feasible values of

$$l_1, u_1, l_2, u_2 \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$$

using 100 independent simulation runs per instance. The outcomes are presented in the following figure 1, with all relevant code included in the appendix.

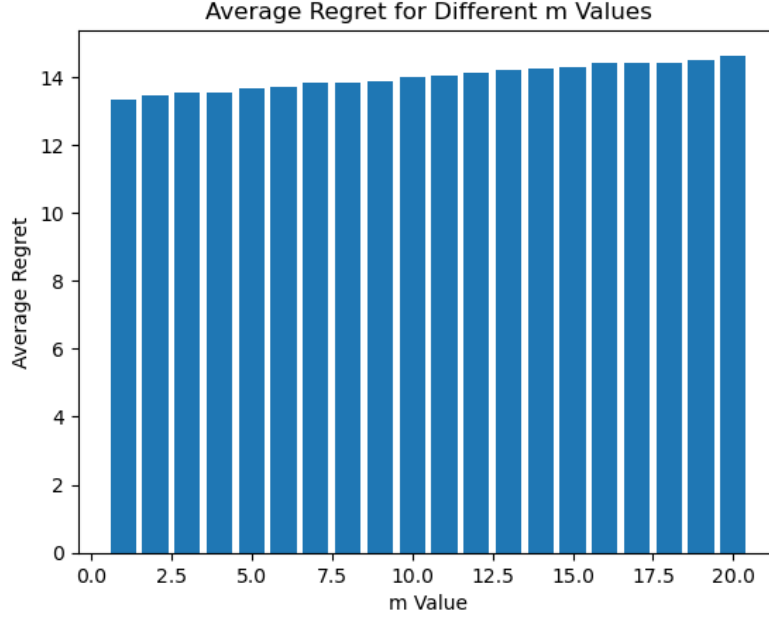


Figure 1: Average Regret for Different m Values.

It is observed that the average regret escalates with an increase in m . Consequently, $m = 1$ will be employed in the subsequent simulation.

To estimate the instance-dependent regret of our policy after n time periods, for each of the 100 feasible instances on the specified grid, we simulate the bandit $M = 1000$ times and compute the realised regret in run j as

$$G_j = n\mu^*(\nu) - \sum_{t=1}^n X_t^{(j)},$$

then estimate the expected regret by $\hat{R}_\nu^\pi(n) = M^{-1} \sum_{j=1}^M G_j$. The figures show the 2.5th and 97.5th percentiles of the realised regrets, giving a central 95% empirical interval for individual runs rather than a confidence interval for the expected regret, as depicted in figure 2, where the vertical axis represents $[l_1, u_1]$, and the horizontal axis represents $[l_2, u_2]$.

	[0.1, 0.3]	[0.1, 0.5]	[0.1, 0.7]	[0.1, 0.9]	[0.3, 0.5]	[0.3, 0.7]	[0.3, 0.9]	[0.5, 0.7]	[0.5, 0.9]
[0.1, 0.3]	0.0020 (-1.8239-1.5505)	0.0435 (4.8432-11.3197)	12.4282 (7.8513-18.8446)	17.2684 (9.7387-24.8384)	13.2305 (10.2912-16.2352)	17.2384 (12.3285-21.9012)	20.1772 (13.0009-27.0069)	20.1404 (14.8504-25.0216)	22.3726 (15.5236-29.2003)
[0.1, 0.5]	0.0094 (4.8964-11.2476)	0.0439 (-3.1379-3.3391)	0.9489 (3.8313-13.3258)	14.8988 (8.2828-20.9358)	0.8164 (5.8561-11.0361)	14.9752 (10.9485-19.2839)	18.9334 (13.3276-26.1942)	19.9076 (15.3985-24.2498)	23.5397 (17.9922-30.1208)
[0.1, 0.7]	13.5251 (7.9920-19.6695)	0.6671 (4.1029-13.1893)	-0.0739 (-5.1559-5.0966)	0.8022 (2.9414-15.0835)	-0.0258 (-3.6542-3.1676)	0.8731 (4.6024-12.9969)	15.9897 (10.2723-21.4530)	15.9388 (12.4991-20.0937)	21.6179 (16.0090-27.2148)
[0.1, 0.9]	17.2982 (9.7236-24.8228)	15.1208 (9.2037-21.8921)	0.8894 (3.6524-14.7901)	0.0241 (-6.2640-6.3083)	0.8158 (3.1988-14.0910)	-0.0315 (-3.0542-4.9121)	0.1207 (3.2684-14.7703)	0.9702 (4.6801-13.2547)	16.9630 (11.8708-21.7112)
[0.3, 0.5]	13.2005 (10.3234-16.5711)	0.4955 (5.8880-11.2962)	0.0428 (-3.5483-3.5532)	0.1076 (3.3058-14.5191)	-0.0143 (-1.6727-1.5893)	0.8149 (5.7989-12.0096)	16.1269 (10.9267-21.2456)	15.9532 (12.9501-18.9684)	21.8941 (16.7832-26.8094)
[0.3, 0.7]	17.2992 (12.2289-22.2928)	18.1132 (11.0079-19.1233)	0.9333 (4.9540-13.2370)	0.0648 (-4.9618-5.2831)	0.7626 (5.6858-12.0290)	-0.0146 (-3.4469-3.2137)	0.0382 (4.4713-13.5607)	0.0344 (6.3248-11.7967)	16.6953 (12.1130-20.8493)
[0.3, 0.9]	20.3055 (13.3822-26.7647)	20.0149 (13.9985-26.1793)	16.0894 (10.4795-21.6441)	0.0381 (3.1327-14.8221)	16.1472 (11.2339-21.2165)	0.0322 (4.8889-13.4828)	0.0431 (-4.6439-4.8200)	0.0888 (-3.4830-3.6465)	9.3828 (5.1393-13.3607)
[0.5, 0.7]	20.1594 (15.1219-25.7154)	19.9098 (15.6953-24.5224)	15.9070 (12.5951-19.7851)	0.0526 (4.8724-13.1256)	16.0460 (12.9189-19.1973)	0.0940 (6.5091-11.6900)	0.0037 (-3.4987-3.6760)	-0.0055 (-2.4198-1.6789)	9.1471 (6.1041-12.1221)
[0.5, 0.9]	22.3974 (16.7780-29.2365)	24.0738 (17.5521-30.5016)	21.7719 (16.4230-27.3649)	16.8622 (11.2349-21.9163)	21.8892 (17.2619-26.9904)	16.8489 (12.5387-20.6833)	0.1973 (5.0622-13.1260)	0.1315 (8.0881-12.1861)	0.0049 (-3.0234-3.2399)
[0.7, 0.9]	23.9355 (17.1038-31.2152)	27.3055 (20.3610-34.2639)	26.5467 (20.4338-32.9472)	22.8941 (17.6027-28.2771)	26.9652 (21.5032-32.6352)	23.0354 (16.6529-27.5040)	17.1254 (13.3335-20.8423)	17.2032 (14.1241-20.3435)	0.0216 (-1.4972-1.6727)

Figure 2: Estimated Mean Regret & Central 95% Empirical Intervals.

From figure 2, we can find that the estimated mean regret ranges from approximately 0 to 28.¹ The estimated regret on the diagonal is approximately zero because the two arms have identical reward distributions there. Off the diagonal, regret depends on the reward means and the sampling probabilities; distance from the diagonal alone does not determine the difference between the arm means.

2.3 Evaluation

For example, if we let $l_1 = 0.7$, $u_1 = 0.9$ and $l_2 = 0.1$, $u_2 = 0.5$, the estimated mean regret is 27.3055, with a central 95% empirical interval for individual runs of (20.2618, 34.3639). In contrast, employing a *random guess policy*, wherein each arm is selected independently with probability 1/2, the instance-dependent regret would manifest as

$$R_{\nu}^{\pi}(n) = 200 \times 0.8 - (100 \times 0.8 + 100 \times 0.3) = 50.$$

For this instance, the estimated regret of our policy is substantially lower than that of uniform random selection.

Furthermore, we also implement both fixed-action policies that use the same action in all time periods as a sanity check. Similarly, we estimate the regret over 1000 simulation runs and give the central 95% empirical intervals.

	[0.1, 0.3]	[0.1, 0.5]	[0.1, 0.7]	[0.1, 0.9]	[0.3, 0.5]	[0.3, 0.7]	[0.3, 0.9]	[0.5, 0.7]	[0.5, 0.9]	[0.7, 0.9]
[0.1, 0.3]	0.0015 (-1.5303-1.5661)	19.9611 (18.3366-21.0529)	40.0211 (38.4368-41.6405)	59.9829 (58.4570-61.6499)	39.9937 (38.3250-41.5935)	60.0004 (58.3767-61.5690)	80.0253 (78.3622-81.6344)	80.0103 (78.4511-81.6339)	99.9729 (98.2342-101.5975)	120.0270 (118.3289-121.6014)
[0.1, 0.5]	0.0007 (-3.0711-3.0824)	-0.0233 (-3.1080-3.0014)	19.9384 (18.7895-23.1040)	39.9694 (38.7202-43.1516)	19.9998 (16.7563-23.0960)	40.0057 (36.7484-43.2240)	60.0469 (56.7885-63.1657)	60.0087 (57.0158-63.0885)	79.9113 (76.0642-83.2966)	99.9558 (96.5632-102.9637)
[0.1, 0.7]	0.0702 (-4.8956-4.9551)	0.0032 (-4.9089-4.7690)	-0.1433 (-5.0392-4.8368)	20.0343 (15.1700-24.9200)	0.2312 (-4.6460-4.7520)	19.9611 (15.3470-24.5731)	40.0795 (36.6200-45.1356)	39.9681 (35.2196-44.6521)	59.9940 (55.1937-65.0702)	80.0140 (75.1939-85.0357)
[0.1, 0.9]	0.0101 (-6.3637-6.3923)	0.0240 (-6.0754-6.5505)	-0.0274 (-6.5073-6.4543)	-0.1885 (-6.8220-6.4596)	-0.2323 (-6.4476-6.7720)	0.0790 (-5.8000-6.4619)	19.9858 (13.8129-26.1350)	20.0372 (13.4829-26.4316)	40.0786 (33.9394-46.3519)	59.9895 (53.4085-65.7888)
[0.3, 0.5]	-0.0053 (-1.6149-1.5662)	-0.0139 (-1.6607-1.5947)	-0.0204 (-1.5341-1.5299)	19.9811 (18.3208-21.5893)	-0.0434 (-1.6134-1.5147)	20.0007 (18.3989-21.5120)	40.0392 (38.4314-41.5945)	40.0279 (38.4707-41.6920)	60.0259 (56.4785-61.6598)	80.0119 (76.4789-81.5459)
[0.3, 0.7]	0.0061 (-3.1580-3.2400)	-0.0261 (-3.1011-3.1169)	0.0330 (-3.0873-3.1047)	-0.0229 (-3.4700-3.4921)	-0.0039 (-3.3143-3.1095)	0.0143 (-3.0619-3.1191)	20.0021 (16.9101-23.2531)	20.0234 (16.9207-23.0102)	39.9701 (36.7794-43.0688)	60.0207 (56.9556-63.4678)
[0.3, 0.9]	0.0058 (-4.8017-4.9250)	0.0117 (-4.9352-4.8545)	0.2056 (-4.8052-5.1639)	-0.0636 (-4.8399-4.6172)	-0.0869 (-4.7469-4.6810)	0.0627 (-4.7269-4.7096)	0.0869 (-4.6759-4.7271)	-0.0433 (-4.9040-4.8658)	19.8750 (14.9021-24.7845)	40.0515 (36.4259-45.0376)
[0.5, 0.7]	-0.0030 (-1.4723-1.6300)	0.0100 (-1.5659-1.4877)	-0.0415 (-1.6544-1.6423)	-0.0124 (-1.6471-1.6084)	0.0123 (-1.6177-1.4745)	-0.0081 (-1.5619-1.5626)	0.0031 (-1.6730-1.6495)	0.0059 (-1.5875-1.6247)	19.9850 (18.4695-21.6612)	40.0282 (36.3495-41.5526)
[0.5, 0.9]	0.0022 (-3.2697-3.1740)	0.0285 (-3.3130-3.2055)	0.0539 (-3.2157-3.0989)	-0.0522 (-3.3090-3.1414)	-0.0218 (-3.4979-3.1420)	0.0901 (-3.0930-3.3034)	-0.0187 (-3.2239-3.6997)	-0.0102 (-3.2585-3.2098)	0.0446 (-3.0805-3.1488)	19.9859 (16.9304-23.0588)
[0.7, 0.9]	-0.0547 (-1.7014-1.3985)	-0.0048 (-1.5560-1.7295)	0.0131 (-1.5162-1.6619)	0.0300 (-1.6071-1.5383)	-0.0226 (-1.4915-1.5533)	-0.0021 (-1.5139-1.6113)	-0.0022 (-1.5140-1.4954)	-0.0289 (-1.6300-1.5295)	-0.0193 (-1.6854-1.6948)	0.0115 (-1.5175-1.5239)

Figure 3: Fixed arm 1: mean regret and central 95% empirical intervals.

	[0.1, 0.3]	[0.1, 0.5]	[0.1, 0.7]	[0.1, 0.9]	[0.3, 0.5]	[0.3, 0.7]	[0.3, 0.9]	[0.5, 0.7]	[0.5, 0.9]	[0.7, 0.9]
[0.1, 0.3]	0.0014 (-1.4705-1.5210)	-0.0098 (-3.2954-3.1845)	-0.0243 (-4.5064-4.7490)	0.0207 (-4.2557-6.2985)	0.0039 (-1.6397-1.0953)	-0.0096 (-3.2080-3.1887)	-0.0337 (-4.9199-4.7968)	0.0071 (-1.4879-1.5060)	0.0277 (-3.2311-3.0264)	0.0068 (-1.6415-1.5703)
[0.1, 0.5]	19.9971 (18.4681-21.5670)	-0.0076 (-3.2374-3.0891)	0.0539 (-4.9039-4.8160)	0.0039 (-6.0570-6.5791)	0.0320 (-1.5680-1.5664)	-0.0204 (-3.2012-3.1494)	-0.1140 (-5.0237-4.8522)	-0.0309 (-1.6094-1.5434)	0.0160 (-3.3099-3.1440)	-0.0071 (-1.6131-1.6780)
[0.1, 0.7]	40.0287 (38.4880-41.6121)	19.9112 (18.8392-22.9033)	0.0640 (-4.2228-6.2957)	0.1871 (-6.2009-6.0453)	-0.0363 (-1.7090-1.5984)	0.0282 (-3.0722-3.2398)	-0.1563 (-5.4501-4.7862)	0.0056 (-1.5421-1.5308)	0.0283 (-2.8705-3.0802)	0.0052 (-1.5373-1.5978)
[0.1, 0.9]	59.9991 (58.3623-61.5460)	40.0090 (37.0693-43.1447)	19.9932 (15.0039-24.7393)	-0.1113 (-6.5562-6.4642)	20.0044 (15.9718-21.6598)	-0.0470 (-3.2552-3.2239)	-0.0805 (-4.8698-4.7543)	0.0071 (-1.5302-1.6220)	-0.0336 (-3.3291-3.0489)	0.0056 (-1.6301-1.6254)
[0.3, 0.5]	40.0099 (38.4825-41.6685)	19.9858 (16.7730-23.1347)	-0.0581 (-4.5404-4.8266)	0.0023 (-6.6380-6.3095)	0.0224 (-1.5185-1.5626)	0.0460 (-3.1699-3.2371)	0.0284 (-5.1479-4.8886)	0.0014 (-1.5376-1.6304)	-0.0204 (-3.1467-3.1162)	0.0251 (-1.5861-1.7094)
[0.3, 0.7]	59.9997 (58.4052-61.5656)	40.0025 (36.7771-43.3379)	19.9220 (15.2119-24.6420)	-0.1107 (-6.2626-6.0864)	19.9798 (18.5438-21.6222)	-0.0100 (-3.1728-3.1771)	-0.0002 (-4.8984-6.0739)	-0.0429 (-1.6040-1.5645)	-0.0117 (-3.0088-3.0295)	-0.0227 (-1.7056-1.6132)
[0.3, 0.9]	79.9873 (78.2455-81.4751)	59.9215 (56.9540-63.2571)	39.9710 (35.0743-44.8740)	20.0914 (13.5036-26.2853)	40.0103 (38.5495-41.6595)	19.9854 (18.9922-23.9917)	0.0044 (-4.8544-4.9676)	-0.0109 (-1.6765-1.5568)	0.0091 (-3.1684-3.0088)	-0.0152 (-1.6071-1.5030)
[0.5, 0.7]	79.9843 (78.4031-81.6262)	60.0501 (56.6969-63.3240)	40.0639 (34.9384-45.0713)	19.9268 (13.7444-26.2056)	40.0089 (38.5831-41.7697)	20.0001 (18.9279-23.9951)	0.0191 (-4.7960-4.8094)	-0.0082 (-1.5500-1.5940)	-0.0728 (-3.2798-3.0576)	-0.0389 (-1.6078-1.6540)
[0.5, 0.9]	100.0050 (98.4651-101.6499)	79.9941 (76.6769-83.1941)	60.0419 (55.2090-65.1475)	39.9871 (33.8314-45.1828)	60.0080 (58.3718-61.5349)	40.0055 (36.8457-43.3888)	20.0422 (15.1902-24.8728)	20.0141 (18.5889-21.5793)	-0.0581 (-3.4530-3.4936)	0.0281 (-1.6209-1.6142)
[0.7, 0.9]	120.0551 (118.5363-121.6032)	100.0119 (96.0874-103.9111)	80.0004 (75.3853-85.0794)	60.0137 (53.6016-66.0033)	79.9607 (76.4640-81.6064)	59.9894 (56.6497-62.8722)	40.0168 (35.0017-44.9931)	40.0156 (38.3780-41.5320)	19.9370 (16.7559-23.0909)	0.0343 (-1.5757-1.6945)

Figure 4: Fixed arm 2: mean regret and central 95% empirical intervals.

Observing these two tables (in figure 3 and figure 4, respectively), the outcomes largely align with anticipated expectations. When arm 1 has a higher expected reward than arm 2, the estimated regrets in the first table are approximately zero,² while the values of regrets are relatively high in the second table. Conversely, should arm 2 exhibit a higher expected reward, always selecting arm 1 would incur significant regret, whereas the estimated regret remains almost zero with fixed selections of arm 2. The exact expected regret of always selecting arm a is $n(\mu^* - \mu_a)$.

The largest estimated mean regret reported is 27.3055, attained when $l_1 = 0.7$, $u_1 = 0.9$, $l_2 = 0.1$, $u_2 = 0.5$. Swapping the arm labels gives the same expected regret by symmetry, although Monte Carlo estimates can differ. To give a reason why we have the *worst-case regret* in this situation, we analyse the expected reward for every step after the first $2m$ rounds. To give an intuitive illustration, we compute the conditional expected reward at each step by

$$\mathbb{E}(X_t | h) = 0.8 \cdot \mathbb{P}(A_t = 1 | h) + 0.3 \cdot \mathbb{P}(A_t = 2 | h)$$

¹When the two arms have the same expected reward, expected regret is zero. Its Monte Carlo estimate can be slightly negative because of simulation noise. Expected regret itself is non-negative.

²Small negative estimates are consistent with simulation noise around zero; they are retained rather than truncated.

$$\begin{aligned}
&= \frac{0.8 \cdot \frac{\sum_{i=1}^{t-1} X_i \mathbb{1}_{\{A_i=1\}}}{\sum_{i=1}^{t-1} \mathbb{1}_{\{A_i=1\}}}}{\frac{\sum_{i=1}^{t-1} X_i \mathbb{1}_{\{A_i=1\}}}{\sum_{i=1}^{t-1} \mathbb{1}_{\{A_i=1\}}} + \frac{\sum_{i=1}^{t-1} X_i \mathbb{1}_{\{A_i=2\}}}{\sum_{i=1}^{t-1} \mathbb{1}_{\{A_i=2\}}}} + \frac{0.3 \cdot \frac{\sum_{i=1}^{t-1} X_i \mathbb{1}_{\{A_i=2\}}}{\sum_{i=1}^{t-1} \mathbb{1}_{\{A_i=2\}}}}{\frac{\sum_{i=1}^{t-1} X_i \mathbb{1}_{\{A_i=1\}}}{\sum_{i=1}^{t-1} \mathbb{1}_{\{A_i=1\}}} + \frac{\sum_{i=1}^{t-1} X_i \mathbb{1}_{\{A_i=2\}}}{\sum_{i=1}^{t-1} \mathbb{1}_{\{A_i=2\}}}} \\
&= \frac{0.8 \cdot \frac{\sum_{i=1}^{t-1} X_i \mathbb{1}_{\{A_i=1\}}}{\sum_{i=1}^{t-1} \mathbb{1}_{\{A_i=1\}}} + 0.3 \cdot \frac{\sum_{i=1}^{t-1} X_i \mathbb{1}_{\{A_i=2\}}}{\sum_{i=1}^{t-1} \mathbb{1}_{\{A_i=2\}}}}{\frac{\sum_{i=1}^{t-1} X_i \mathbb{1}_{\{A_i=1\}}}{\sum_{i=1}^{t-1} \mathbb{1}_{\{A_i=1\}}} + \frac{\sum_{i=1}^{t-1} X_i \mathbb{1}_{\{A_i=2\}}}{\sum_{i=1}^{t-1} \mathbb{1}_{\{A_i=2\}}}}.
\end{aligned}$$

Thus we can plot the expected reward values as follows:

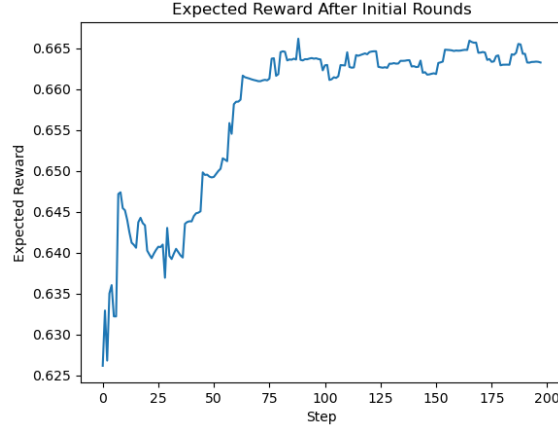


Figure 5: Conditional Expected Reward after the Initial Rounds.

From the figure 5, we find that the conditional expected reward curve becomes stable after 75 rounds, which arrives at a value around 0.664. As both sample means approach their true values, the conditional expected reward approaches

$$\frac{0.8^2}{0.8 + 0.3} + \frac{0.3^2}{0.8 + 0.3} \approx 0.6636,$$

which conforms to what the plot shows. We also notice that the limit is still below the optimal expected reward per round, 0.8. So in the example, the reasons that lead to the worst-case regret may include:

- (i) The initial exploration and uncertainty in the sample means contribute to regret. This plot alone does not establish that a larger gap between the arms causes slower convergence.
- (ii) Even after the estimates become accurate, the policy continues to select the inferior arm with positive probability. In this instance, its limiting probability is $0.3/(0.8 + 0.3)$, giving an asymptotic regret per round of $0.8 - 0.6636 \approx 0.1364$. This persistent exploration is the main long-run source of regret.

Remark. The simulations show that our policy reduces regret relative to uniform random selection in the example above. Unlike an explore-then-commit policy, it does not permanently commit after the initial observations. However, no direct comparison with other explore-then-commit policies was performed, and persistent exploration prevents the policy from approaching the optimal expected reward per round. Therefore, we believe that some improvements can be made to our policy. For the problem addressed in this report, one possible improvement is to set a threshold α , for example, let $\alpha = 0.2$. When both arms have been sampled at least 30 times during the bandit playing process, if their estimated expectation difference exceeds this threshold, we could choose the arm with the higher payoff in the following selections.

3 Conclusion

Throughout this report, we delineate a policy to increase cumulative rewards for a two-armed bandit over $n = 200$ time intervals. Furthermore, through assessment, we demonstrate the efficacy of our approach in diminishing regrets, whilst also addressing the prevailing limitations of our methodology and suggesting avenues for future enhancements.

A Codes

```
# In[1]:

import numpy as np

class MultiArmedBandit():
    def __init__(self, num_arms, lower_bounds, upper_bounds):
        self.num_arms = num_arms
        self.lower_bounds = lower_bounds
        self.upper_bounds = upper_bounds

    def generate_reward(self, action):
        if action < 0 or action >= self.num_arms:
            raise ValueError("Invalid action")
        return np.random.uniform(self.lower_bounds[action],
                                self.upper_bounds[action])

    def find_optimal_arm(self):
        expected_rewards = [(lb + ub) / 2 for lb, ub in
                             zip(self.lower_bounds, self.upper_bounds)]
        return np.argmax(expected_rewards)

    def max_expected_reward(self):
        expected_rewards = [(lb + ub) / 2 for lb, ub in
                             zip(self.lower_bounds, self.upper_bounds)]
        return max(expected_rewards)

# In[2]:

def play_bandit(bandit, m, n):
    total_rewards = 0
    num_arms = bandit.num_arms
    num_initial_rounds = m * num_arms
    num_total_rounds = n
    regret = 0

    arm_counts = [0] * num_arms
    arm_rewards = [0] * num_arms

    for i in range(num_arms):
        action = i
        for _ in range(m):
            reward = bandit.generate_reward(action)
            total_rewards += reward
            arm_counts[action] += 1
            arm_rewards[action] += reward

    for _ in range(num_total_rounds - num_initial_rounds):
        arm_means = [reward / count if count > 0 else 0 for reward, count in
                     zip(arm_rewards, arm_counts)]
        total_mean = sum(arm_means)
        action_probabilities = (
            [mean / total_mean for mean in arm_means]
            if total_mean > 0 else [1 / num_arms] * num_arms
        )
        next_action = np.random.choice(range(num_arms), p=action_probabilities)
```

```
        reward = bandit.generate_reward(next_action)
        total_rewards += reward
        arm_counts[next_action] += 1
        arm_rewards[next_action] += reward

    regret = n * bandit.max_expected_reward() - total_rewards
    return regret

# In[11]:

import itertools
import matplotlib.pyplot as plt

# Define the values for arm bounds
arm_bounds_values = [0.1, 0.3, 0.5, 0.7, 0.9]

# Generate all combinations of arm bounds
arm_combinations = list(itertools.product(arm_bounds_values, arm_bounds_values))

# Filter combinations where the lower bound is less than the upper bound
arm_combinations = [comb for comb in arm_combinations if comb[0] < comb[1]]

# Define a range of m values to test, limited to 1 to 20
m_values = list(range(1, 21))

# Number of iterations for each bandit
num_iterations = 100

# Initialize a dictionary to store average regrets for each m value
avg_regrets_per_m = {m: [] for m in m_values}

# Iterate over each m value
for m in m_values:
    # Initialize a list to store regrets for all arm combinations
    all_regrets = []

    # Iterate over each combination of arm bounds
    for (arm1_lower, arm1_upper), (arm2_lower, arm2_upper) in (
        itertools.product(arm_combinations, repeat=2)
    ):
        # Create MultiArmedBandit object for the current combination of
        # arm bounds
        arm1_bounds = [arm1_lower, arm1_upper]
        arm2_bounds = [arm2_lower, arm2_upper]
        lower_bounds = [arm1_lower, arm2_lower]
        upper_bounds = [arm1_upper, arm2_upper]
        bandit = MultiArmedBandit(2, lower_bounds, upper_bounds)

        # Calculate regrets for the bandit over 100 iterations
        regrets = [play_bandit(bandit, m, 200) for _ in range(num_iterations)]
        all_regrets.extend(regrets)

    # Calculate average regret for the current m value
    avg_regret = np.mean(all_regrets)
    avg_regrets_per_m[m] = avg_regret

# Plot the bar chart
plt.bar(avg_regrets_per_m.keys(), avg_regrets_per_m.values())
plt.xlabel('m Value')
plt.ylabel('Average Regret')
plt.title('Average Regret for Different m Values')
plt.show()

# In[8]:

from tabulate import tabulate
```

```
import itertools

# Define the values for arm bounds
arm_bounds_values = [0.1, 0.3, 0.5, 0.7, 0.9]

# Generate all combinations of arm bounds
arm_combinations = list(itertools.product(arm_bounds_values, arm_bounds_values))

# Filter combinations where the lower bound is less than the upper bound
arm_combinations = [comb for comb in arm_combinations if comb[0] < comb[1]]

# Initialize a 10x10 table for mean regrets and empirical intervals
avg_regrets_table = [["" for _ in range(10)] for _ in range(10)]

# Number of iterations for each bandit
num_iterations = 1000

# Iterate over each combination of arm bounds
for i, (arm1_lower, arm1_upper) in enumerate(arm_combinations):
    for j, (arm2_lower, arm2_upper) in enumerate(arm_combinations):
        # Create MultiArmedBandit object for the current combination of
        # arm bounds
        arm1_bounds = [arm1_lower, arm1_upper]
        arm2_bounds = [arm2_lower, arm2_upper]
        lower_bounds = [arm1_lower, arm2_lower]
        upper_bounds = [arm1_upper, arm2_upper]
        bandit = MultiArmedBandit(2, lower_bounds, upper_bounds)

        # Calculate regrets for the bandit over 1000 iterations
        regrets = [play_bandit(bandit, 1, 200) for _ in range(num_iterations)]

        # Calculate mean regret and the central 95% empirical interval
        avg_regret = np.mean(regrets)
        empirical_interval = np.percentile(regrets, [2.5, 97.5])

        # Store mean regret and the empirical interval in the table
        avg_regrets_table[i][j] = "{:.4f} ({:.4f}-{:.4f})".format(avg_regret,
                                                                empirical_interval[0], empirical_interval[1])

# Convert the table data to a list of lists for tabulate
table_data = []
for (arm1_lower, arm1_upper), row in zip(arm_combinations, avg_regrets_table):
    table_row = [f"[{arm1_lower}, {arm1_upper}]" + row]
    table_data.append(table_row)

# Print the table
headers = [""] + [f"[{lower}, {upper}]" for lower, upper in arm_combinations]
print(tabulate(table_data, headers=headers, tablefmt="grid"))

# In[14]:

def sanity_check(bandit, arm_index, n):
    total_rewards = 0
    num_arms = bandit.num_arms
    regret_arm = 0

    # Calculate total rewards and regret for one arm
    for _ in range(n):
        reward = bandit.generate_reward(arm_index)
        total_rewards += reward
    regret_arm = n * bandit.max_expected_reward() - total_rewards

    return regret_arm

# In[15]:
```

```
# Define the values for arm bounds
arm_bounds_values = [0.1, 0.3, 0.5, 0.7, 0.9]

# Generate all combinations of arm bounds
arm_combinations = list(itertools.product(arm_bounds_values, arm_bounds_values))

# Filter combinations where the lower bound is less than the upper bound
arm_combinations = [comb for comb in arm_combinations if comb[0] < comb[1]]

# Initialize a 10x10 table for mean regrets and empirical intervals
avg_regrets_table = [["" for _ in range(10)] for _ in range(10)]

# Number of iterations for each bandit
num_iterations = 1000

# Iterate over each combination of arm bounds
for i, (arm1_lower, arm1_upper) in enumerate(arm_combinations):
    for j, (arm2_lower, arm2_upper) in enumerate(arm_combinations):
        # Create MultiArmedBandit object for the current combination of
        # arm bounds
        arm1_bounds = [arm1_lower, arm1_upper]
        arm2_bounds = [arm2_lower, arm2_upper]
        lower_bounds = [arm1_lower, arm2_lower]
        upper_bounds = [arm1_upper, arm2_upper]
        bandit = MultiArmedBandit(2, lower_bounds, upper_bounds)

        # Calculate regrets for the bandit over 1000 iterations
        regrets = [sanity_check(bandit, 0, 200) for _ in range(num_iterations)]

        # Calculate mean regret and the central 95% empirical interval
        avg_regret = np.mean(regrets)
        empirical_interval = np.percentile(regrets, [2.5, 97.5])

        # Store mean regret and the empirical interval in the table
        avg_regrets_table[i][j] = "{:.4f} ({:.4f}-{:.4f})".format(avg_regret,
                                                                empirical_interval[0], empirical_interval[1])

# Convert the table data to a list of lists for tabulate
table_data = []
for (arm1_lower, arm1_upper), row in zip(arm_combinations, avg_regrets_table):
    table_row = [f"[{arm1_lower}, {arm1_upper}]" + row]
    table_data.append(table_row)

# Print the table
headers = [""] + [f"[{lower}, {upper}]" for lower, upper in arm_combinations]
print(tabulate(table_data, headers=headers, tablefmt="grid"))
```

In[16]:

```
# Define the values for arm bounds
arm_bounds_values = [0.1, 0.3, 0.5, 0.7, 0.9]

# Generate all combinations of arm bounds
arm_combinations = list(itertools.product(arm_bounds_values, arm_bounds_values))

# Filter combinations where the lower bound is less than the upper bound
arm_combinations = [comb for comb in arm_combinations if comb[0] < comb[1]]

# Initialize a 10x10 table for mean regrets and empirical intervals
avg_regrets_table = [["" for _ in range(10)] for _ in range(10)]

# Number of iterations for each bandit
num_iterations = 1000

# Iterate over each combination of arm bounds
for i, (arm1_lower, arm1_upper) in enumerate(arm_combinations):
    for j, (arm2_lower, arm2_upper) in enumerate(arm_combinations):
        # Create MultiArmedBandit object for the current combination of
```

```
# arm bounds
arm1_bounds = [arm1_lower, arm1_upper]
arm2_bounds = [arm2_lower, arm2_upper]
lower_bounds = [arm1_lower, arm2_lower]
upper_bounds = [arm1_upper, arm2_upper]
bandit = MultiArmedBandit(2, lower_bounds, upper_bounds)

# Calculate regrets for the bandit over 1000 iterations
regrets = [sanity_check(bandit, 1, 200) for _ in range(num_iterations)]

# Calculate mean regret and the central 95% empirical interval
avg_regret = np.mean(regrets)
empirical_interval = np.percentile(regrets, [2.5, 97.5])

# Store mean regret and the empirical interval in the table
avg_regrets_table[i][j] = "{:.4f} ({:.4f}-{:.4f})".format(avg_regret,
                                                         empirical_interval[0], empirical_interval[1])

# Convert the table data to a list of lists for tabulate
table_data = []
for (arm1_lower, arm1_upper), row in zip(arm_combinations, avg_regrets_table):
    table_row = [f"[{arm1_lower}, {arm1_upper}]" + row]
    table_data.append(table_row)

# Print the table
headers = [""] + [f"[{lower}, {upper}]" for lower, upper in arm_combinations]
print(tabulate(table_data, headers=headers, tablefmt="grid"))

# In[11]:

import matplotlib.pyplot as plt

def plot_expected_rewards(bandit, m, n):
    num_arms = bandit.num_arms
    num_initial_rounds = m * num_arms
    num_total_rounds = n

    arm_counts = [0] * num_arms
    arm_rewards = [0] * num_arms
    expected_rewards = []
    arm_expected_rewards = [(lb + ub) / 2 for lb, ub in zip(bandit.lower_bounds,
                                                           bandit.upper_bounds)]

    for i in range(num_arms):
        action = i
        for _ in range(m):
            reward = bandit.generate_reward(action)
            arm_counts[action] += 1
            arm_rewards[action] += reward

    for _ in range(num_total_rounds - num_initial_rounds):
        arm_means = [reward / count if count > 0 else 0 for reward, count in
                     zip(arm_rewards, arm_counts)]
        total_mean = sum(arm_means)
        action_probabilities = (
            [mean / total_mean for mean in arm_means]
            if total_mean > 0 else [1 / num_arms] * num_arms
        )
        expected_reward = np.dot(arm_expected_rewards, action_probabilities)
        expected_rewards.append(expected_reward)

        next_action = np.random.choice(range(num_arms), p=action_probabilities)
        reward = bandit.generate_reward(next_action)
        arm_counts[next_action] += 1
        arm_rewards[next_action] += reward

    plt.plot(range(num_total_rounds - num_initial_rounds), expected_rewards)
    plt.xlabel('Step')
```

```
plt.ylabel('Expected Reward')
plt.title('Expected Reward After Initial Rounds')
plt.show()

num_arms = 2
lower_bounds = [0.7, 0.1]
upper_bounds = [0.9, 0.5]

bandit = MultiArmedBandit(num_arms, lower_bounds, upper_bounds)

m = 1
n = 200

plot_expected_rewards(bandit, m, n)
```

References

- [1] Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge: Cambridge University Press, 2020. ISBN: 9781108486828. DOI: 10.1017/9781108571401.